Linux Text Processing Masterclass: grep, sed & awk

A step-by-step DevOps guide to searching logs, modifying configurations, and extracting metrics from the terminal—with every flag, symbol, and command explained in plain English.

1. The "Big Three" Mental Model: When to Use Which Tool

In DevOps, almost everything is text: application logs, Kubernetes manifests, Nginx configs, CSV reports, and command outputs. While **`grep`**, **`sed`**, and **`awk`** can sometimes overlap, each tool has one superpower. Think of them using everyday desktop analogies:

Tool	Everyday Analogy	Core Superpower	Typical DevOps Use Case
`grep`	**Ctrl + F (Find)** in a document	**Filter rows**: Finds lines that match a specific word or pattern and hides everything else.	Finding all **`ERROR`** or **`500`** lines inside a 2 GB production log file.
`sed`	**Ctrl + H (Find & Replace)** in Word	**Transform text**: Replaces words, deletes lines, or inserts text directly inside files or streams.	Updating an image tag or IP address inside a **`config.yaml`** file during a CI/CD build.
`awk`	**Excel / Google Sheets** in the terminal	**Extract columns & do math**: Splits each line into numbered columns (**`$1`**, **`$2`**), compares numbers, and calculates totals.	Pulling only the IP address and response time columns from an access log, or summing memory usage.

2. Regular Expressions (Regex) Made Effortless

Before diving into the commands, you need to know **Regular Expressions (Regex)**. Regex is simply a mini-language of special symbols ("wildcards") that let you search for *patterns* (like "any number" or "lines starting with #") instead of just fixed words. All three tools—**`grep`**, **`sed`**, and **`awk`**—use these exact same symbols:

Symbol	Plain-English Meaning	Simple Pattern Example	What It Matches in Practice
`^`	**Start of a line**	**`^ERROR`**	Matches lines that *begin* with **`ERROR`** (ignores **`ERROR`** in the middle of a line).
`$`	**End of a line**	**`failed$`**	Matches lines that *end* with the word **`failed`**. (Tip: **`^$`** matches a completely blank line!).

Symbol	Plain-English Meaning	Simple Pattern Example	What It Matches in Practice
`.`	**Any single character** (letter, number, or symbol)	`c.t`	Matches `cat`, `cut`, `c9t`, or `c-t`, but not `ct`.
`*`	**Zero or more** of the previous character	`.*`	Because `.` means "any character" and `*` means "any amount", `.*` means **"anything at all"**.
`+`	**One or more** of the previous character	`[0-9]+`	Matches any whole number (one or more digits in a row, like `4`, `200`, or `8080`).
`?`	**Optional** (zero or one of the previous character)	`https?`	The `s` is optional—matches both `http` and `https`.
`[ ]`	**Any ONE character** inside the brackets	`[0-9]` or `[a-zA-Z]`	`[0-9]` matches any single digit; `50[0234]` matches `500`, `502`, `503`, or `504`.
`[^ ]`	**NOT** any character inside the brackets	`[^0-9]`	When `^` is *inside* brackets, it means "NOT"—matches anything that is not a digit.
`|`	**OR** (matches either the left or right side)	`ERROR|WARN|FATAL`	Matches lines containing `ERROR`, **or** `WARN`, **or** `FATAL`.
`\`	**Escape symbol** (treats a special symbol as normal text)	`10\.0\.0\.1`	Because `.` normally means "any character", `\.` tells Linux to match an actual literal dot.

3. Masterclass Part 1: `grep` (Searching & Filtering Text)

What `grep` stands for: *Global Regular Expression Print*. In simple words: it reads a file (or command output) line by line, checks if the line matches your search word, and prints only the matching lines.

Basic Formula: `grep [options] "search_pattern" filename`

Every Essential `grep` Flag Explained Simply

Flag	Easy Way to Remember	What It Does in Plain Words
`-i`	**I**gnore case	Treats uppercase and lowercase letters the same (searching `"error"` will match `error`, `Error`, and `ERROR`).
`-v`	In**v**ert match	Shows every line that does **NOT** contain your search word. Great for hiding noisy health-check logs!
`-n`	Line **N**umber	Prints the exact line number in front of every matching line so you know where to find it in the file.

Flag	Easy Way to Remember	What It Does in Plain Words
`-c`	**C**ount only	Instead of printing the matching lines, it just prints a single number showing *how many* lines matched.
`-r` / `-R`	**R**ecursive	Searches inside an entire folder and all of its subfolders at once.
`-l`	File **L**ist only	Prints only the *filenames* that contain the match, rather than printing the matching lines themselves.
`-w`	Whole **W**ord only	Matches only the exact standalone word (searching `-w "err"` matches `err`, but will NOT match `error`).
`-o`	**O**nly matching part	Prints *only* the exact matched word/pattern rather than printing the entire line it appeared on.
`-E`	**E**xtended Regex	Unlocks modern regex symbols like `|` (OR), `+` (one or more), and `?` without needing backslashes.
`-A N`	**A**fter context	Prints the matching line **plus N** lines *after* it (crucial for seeing stack traces after an error!).
`-B N`	**B**efore context	Prints the matching line **plus N** lines *before* it (to see what happened right before a crash).
`-C N`	**C**enter / Context	Prints **N** lines *before* AND **N** lines *after* the matching line.

Step-by-Step `grep` Command Breakdowns

Command	Word-by-Word Explanation in Simple English
`grep -in "timeout" /var/log/app.log`	Search inside `/var/log/app.log` for the word `timeout`, ignoring uppercase/lowercase (`-i`), and show the line number (`-n`) for every match.
`grep -E "500|502|503" access.log`	Use Extended Regex (`-E`) to find all lines in `access.log` that contain `500` **OR** `502` **OR** `503`.
`grep -A 10 "OutOfMemoryException" app.log`	Find `OutOfMemoryException` in `app.log` and also print the **10 lines immediately after it** (`-A 10`) so you can read the full Java/Python stack trace.
`grep -rnw "DB_PASSWORD" /etc/app/`	Search through all files inside the `/etc/app/` folder and its subfolders (`-r`), match the exact whole word `DB_PASSWORD` (`-w`), and show line numbers (`-n`).
`grep -v "^#" nginx.conf | grep -v "^$"`	Read `nginx.conf`, hide all comment lines that start with `#` (`-v "^#"`), and then hide all completely blank lines (`-v "^$"`). You see only active configuration!
`grep -Eo "[0-9]{1,3}(\.[0-9]{1,3}){3}" app.log`	Extract and print **only** (`-o`) the IPv4 addresses (four groups of 1–3 digits separated by dots) found anywhere inside `app.log`.

4. Masterclass Part 2: sed (Stream Editor — Find, Replace & Delete)

What sed stands for: *Stream Editor*. Think of **sed** as a conveyor belt: text flows in one line at a time, **sed** performs an edit (like replacing a word or deleting a line), and the modified text flows out.

Crucial Safety Rule: By default, **sed** only prints the changed result to your screen—it **does not change your actual file** unless you pass the **-i** (in-place) flag! This lets you safely test your command before saving changes.

The Anatomy of a sed Substitute Command

90% of the time in DevOps, you will use **sed**'s substitute command: **sed 's/OLD/NEW/g' file.txt**

- **s**: Tells **sed** to perform a **Substitute** (Find & Replace) action.
- **/ (Delimiters)**: The three forward slashes separate the search word from the replacement word. *Pro Tip:* If you are replacing file paths like **/var/www** that contain slashes, you can use **|** or **#** instead so you don't go blind escaping slashes! Example: **sed 's|/var/www|/opt/app|g'**.
- **OLD**: The word or regex pattern you want to find.
- **NEW**: The replacement text you want to put in its place.
- **g (Global flag)**: Means "replace **every** match on the line." Without **g**, **sed** only replaces the *very first* match on each line and ignores the rest of that line!

Every Essential sed Flag & Operator Explained Simply

Flag / Action	Easy Way to Remember	What It Does in Plain Words
-i	**I**n-place edit	Saves the changes directly into the file instead of just printing them to the screen.
-i.bak	**I**n-place with **Bak**up	Automatically creates a backup copy (**file.txt.bak**) before modifying the original file—a lifesaver in production!
-n	**N**o auto-print	Stops **sed** from printing every line by default. Used together with **p** to print only specific line ranges.
-E	**E**xtended Regex	Lets you use clean regex symbols like **+**, **?**, **|**, and capture groups **( )** without backslashes.
-e	**E**xecute multiple	Allows you to chain multiple **sed** edits in a single command (e.g., **sed -e 's/a/b/' -e 's/c/d/'**).
d	**D**elete line	Deletes the entire line if it matches your line number or pattern (e.g., **'/DEBUG/d'** deletes all DEBUG lines).
p	**P**rint line	Prints the matched line (almost always paired with **-n**, like **sed -n '10,20p'** to print lines 10 to 20).
&	Matched text placeholder	In the replacement box, **&** represents whatever text was just matched (e.g., **'s/[0-9]+/[&]/g'** wraps numbers in brackets).

Step-by-Step sed Command Breakdowns

Command	Word-by-Word Explanation in Simple English
`sed -i.bak 's/localhost/10.0.1.25/g' .env`	Create a backup file called **`.env.bak`**, then replace every single occurrence (**`g`**) of **`localhost`** with **`10.0.1.25`** directly inside **`.env`** (**`-i`**).
`sed -i 's|/var/log/old|/mnt/logs/new|g' app.conf`	Using **`|`** as the separator (so we don't have to escape **`/`**), replace the folder path **`/var/log/old`** with **`/mnt/logs/new`** directly in **`app.conf`**.
`sed -n '100,150p' /var/log/syslog`	Turn off normal printing (**`-n`**) and print (**`p`**) **only lines 100 through 150** of **`/var/log/syslog`**. Ideal for inspecting a specific slice of a huge file.
`sed '/^#/d; /^$/d' config.yaml`	Perform two delete actions separated by a semicolon: first delete every line starting with **`#`** (**`/^#/d`**), then delete every empty line (**`/^$/d`**).
`sed '1d; $d' report.csv`	Delete the very first line/header (**`1d`**) and delete the very last line/footer (**`$d`**) from **`report.csv`**.
`sed -i '/^Port /s/22/2222/' /etc/ssh/sshd_config`	Find only the line that starts with **`Port `**, and on that specific line, replace **22** with **2222**. Other lines containing **22** are left untouched!
`sed -n '/14:00:00/,/14:05:00/p' app.log`	Extract and print (**`-n ... p`**) every log line between the timestamp **`14:00:00`** and **`14:05:00`**.

5. Masterclass Part 3: awk (Columns, Filtering & Math)

What `awk` stands for: Named after its three creators (*Aho, Weinberger, and Kernighan*). Unlike **`grep`** and **`sed`** which look at whole lines, **`awk`** automatically splits every line into **columns** (called *fields*) using spaces or tabs as the separator.

How awk Sees a Line of Text

Imagine a line in a web server log: **`192.168.1.50 GET /api/orders 200 45ms`**

- **`$0`**: Represents the **entire line** (**`192.168.1.50 GET /api/orders 200 45ms`**).
- **`$1`**: The **1st column** (**`192.168.1.50`**).
- **`$2`**: The **2nd column** (**`GET`**).
- **`$3`**: The **3rd column** (**`/api/orders`**).
- **`$4`**: The **4th column** (**`200`**).
- **`$NF`**: The **very last column** on the line (**`45ms`**), no matter how many columns exist!

The Structure of an awk Command

Full formula: **`awk -F',' 'BEGIN { prep } condition { action } END { summary }' file`**

- **`BEGIN { ... }`** *(Optional)*: Runs **once at the very start** before reading any lines (great for printing table headers or setting variables).
- **`condition { action }`**: Runs on **every single line**. If the line meets the **`condition`** (e.g., **`$4 == 500`**), **`awk`** runs the **`{ action }`** (e.g., **`{ print $1 }`**). If you leave out the condition, the action runs on every line.
- **`END { ... }`** *(Optional)*: Runs **once at the very end** after all lines have been read (great for printing totals or averages).

Every Essential awk Variable & Flag Explained Simply

Flag / Variable	Easy Way to Remember	What It Does in Plain Words
`-F`	**F**ield separator	Tells **awk** how columns are separated if they aren't separated by spaces (e.g., **`-F':'`** for **`/etc/passwd`** or **`-F','`** for CSV files).
`$1, $2, $3...`	Column number	Refers to the 1st, 2nd, 3rd column of the current line.
`$0`	Whole line	Refers to the entire current line (all columns together).
`NR`	**N**umber of **R**ow	Holds the current line number (**1** on the first line, **2** on the second line). Useful for skipping CSV headers with **`NR > 1`**!
`NF` / **`$NF`**	**N**umber of **F**ields	**`NF`** is the count of columns on the line; putting a dollar sign in front (**`$NF`**) gives you the value of the **last column**.
`OFS`	**O**utput **F**ield **S**eparator	Controls what character is printed between columns when you use a comma in **`print $1, $2`** (defaults to a single space).
`~` / **`!~`**	Matches / Doesn't match	Checks if a specific column matches a regex pattern (e.g., **`$3 ~ /^\/api/`** checks if column 3 starts with **`/api`**).

Step-by-Step awk Command Breakdowns

Command	Word-by-Word Explanation in Simple English
`awk '{print $1, $4}' access.log`	Read **`access.log`** and print only the 1st column (IP address) and 4th column (status code) for every line, separated by a space.
`awk -F':' '{print $1, $6}' /etc/passwd`	Split columns using a colon (**`-F':'`**), then print the 1st column (username) and 6th column (home directory) from **`/etc/passwd`**.
`awk 'NR > 1 && $3 > 80 {print $1, $3}' metrics.csv`	Skip the first header row (**`NR > 1`**), check if column 3 is greater than 80 (**`$3 > 80`**), and if so, print columns 1 and 3.

Command	Word-by-Word Explanation in Simple English
`awk '$9 == 500 {print $1, $7}' nginx.log`	Look at the 9th column (HTTP status code); if it equals **500**, print the visitor's IP address (**$1**) and the URL they requested (**$7**).
`awk '{sum += $10} END {print "Total Bytes:", sum}' access.log`	For every line, add the 10th column (response size in bytes) to a running total called **sum**. At the **END** of the file, print the grand total.
`awk '{count[$9]++} END {for (code in count) print code, count[code]}' access.log`	Keep a tally of every unique HTTP status code in column 9 (**count[$9]++**). At the end, loop through and print each status code alongside how many times it occurred!

6. Power Pipelines: Combining grep, sed & awk in Production

The real magic in DevOps happens when you connect these tools together using the pipe symbol (`|`), passing the output of one tool directly into the next.

Scenario 1: Find the Top 5 IP Addresses Causing 500 Errors

```
grep " 500 " /var/log/nginx/access.log | awk '{print $1}' | sort | uniq -c | sort -nr | head -n 5
```

- **`grep " 500 " ...`**: Filters the massive log file down to only lines with HTTP status **500**.
- **`awk '{print $1}'`**: Grabs only the 1st column (the client IP address) from those error lines.
- **`sort | uniq -c`**: Groups identical IP addresses together and counts how many times each IP appeared.
- **`sort -nr | head -n 5`**: Sorts the counts from highest to lowest (**-nr**) and displays the top 5 offenders.

Scenario 2: List All Disk Partitions Over 80% Full

```
df -h | sed '1d; s/%//g' | awk '$5 > 80 {print "ALERT:", $6, "is at", $5"%"}'
```

- **`df -h`**: Lists disk space usage across all mounted partitions.
- **`sed '1d; s/%//g'`**: Deletes the top header row (**1d**) and strips the **%** sign off the numbers (**s/%//g**) so **awk** can compare them mathematically.
- **`awk '$5 > 80 {print ...}'`**: Checks if column 5 (use percentage) is over **80**, and prints a clean alert message with the mount path (**$6**).

Scenario 3: Extract Clean Config Settings & Mask Passwords for Sharing

```
grep -Ev "^#|^$" app.env | sed -E 's/(PASSWORD|SECRET)=.*/\1=********/' | awk -F'=' '{printf "%-22s : %s\n", $1, $2}'
```

- **`grep -Ev "^#|^$" app.env`**: Strips out all comment lines starting with **#** and all blank lines.
- **`sed -E 's/(PASSWORD|SECRET)=.*/\1=********/'`**: Finds any key containing **PASSWORD** or **SECRET** and masks its value with ******** so you can safely paste it into a ticket or Slack.

- **`awk -F'=' '{printf ...}'`**: Splits by the = sign and formats the keys and values into a neatly aligned two-column table.

Scenario 4: Calculate Average API Response Time from Logs

```
grep "POST /api/checkout" app.log | sed 's/ms$//' | awk '{sum += $NF; n++} END {if (n > 0) print "Avg Latency:", sum/n, "ms"}'
```

- **`grep "POST /api/checkout" app.log`**: Isolates only log lines for the checkout API endpoint.
- **`sed 's/ms$//'`**: Removes the trailing **`ms`** text from the last column (e.g., turns **`140ms`** into **`140`**) so **`awk`** can treat it as a number.
- **`awk '{sum += $NF; n++} END { ... }'`**: Adds the last column (**`$NF`**) to **`sum`**, counts the number of requests (**`n++`**), and divides **`sum/n`** at the end to print the exact average latency.

Scenario 5: Find High-Memory Processes & Format a Clean Report

```
ps aux | awk 'NR > 1 && $4 > 5.0 {printf "PID: %-8s | RAM: %-5s%% | CMD: %s\n", $2, $4, $11}' | sort -k5 -nr
```

- **`ps aux`**: Lists all currently running processes on the server.
- **`awk 'NR > 1 && $4 > 5.0 { ... }'`**: Skips the header line (**`NR > 1`**), filters for processes using more than **`5.0%`** RAM (**`$4 > 5.0`**), and formats the PID (**`$2`**), RAM % (**`$4`**), and command name (**`$11`**).
- **`sort -k5 -nr`**: Sorts the output by the 5th column (the RAM percentage) from highest to lowest.